

From Content to Context

Transforming legacy applications into AI operating systems — software that stops waiting to be queried and starts anticipating intent.

• WRITTEN BY **Karlissa Cole**

FORMAT

Illustrated ebook

FOR

Product leaders & builders

FIGURES

5 system visuals

PULL · STATIC CONTENT

PUSH · CONTINUOUS CONTEXT



00 Intro — The death of the pull-based dashboard

02 The proactive UI/UX shift + UX script

04 The transformation playbook

01 Architecting the contextual data model

03 Engineering trust & the autonomy matrix

05 Becoming an agentic PM + references

The Death of the Pull-Based Dashboard

Why static content is a cognitive tax — and what replaces it

The traditional dashboard rests on a quietly flawed assumption: that the person staring at it has the time, the energy, and the analytical training to turn raw data into a decision. Every chart is an invitation to do unpaid work. Every filter is a small tax on attention. This is the **pull** model of software — the system sits inert until a human arrives to extract meaning from it.

For two decades that was the only model we had. The ground has now shifted. In his 2025 talk *Software Is Changing (Again)*, Andrej Karpathy described a progression from Software 1.0 (explicit, hand-written code) to Software 2.0 (learned neural-network weights) to **Software 3.0**, in which large language models become a new programmable layer — effectively a new kind of operating system, with the context window as its working memory. ^[1] Andreessen Horowitz frames the consequence bluntly: AI is becoming "**something that does**" rather than something you ask, collapsing the distance between a user's intent and its execution. ^[2]

"AI is moving from a tool you consult to a system that can understand intent and take action."

A16Z — BIG IDEAS 2026: THE AGENTIC INTERFACE

That collapse is the whole game. When software holds continuous **context** instead of serving static **content**, it can absorb the analytical burden the dashboard used to offload onto us. It stops waiting. It anticipates. The pull-based dashboard isn't being redesigned — it is being retired, replaced by **push-based** systems that surface the right action at the moment it becomes relevant.

Nielsen Norman Group calls this the first genuinely new interaction paradigm in sixty years: a shift from **command-based** interaction, where users specify exactly how to perform each step, to **intent-based** outcome specification, where users state what they want and the system works out the rest. ^[3] This guide is a field manual for the product managers who will build that transition — not by bolting a chatbot onto a legacy app, but by re-architecting the app itself around three pillars: context, proactivity, and trust.

Evolution: Static → Interactive → Agentic

Metric: cognitive friction

Shift: pull → push

● FIGURE 01 – THE SOFTWARE EVOLUTION TIMELINE

Three eras of software, one direction of travel

Each era moved the analytical burden further off the user and onto the system. The horizontal axis is not time alone — it is the migration from **pull** (the user does the work) to **push** (the system anticipates), driven by ever-richer continuous context.

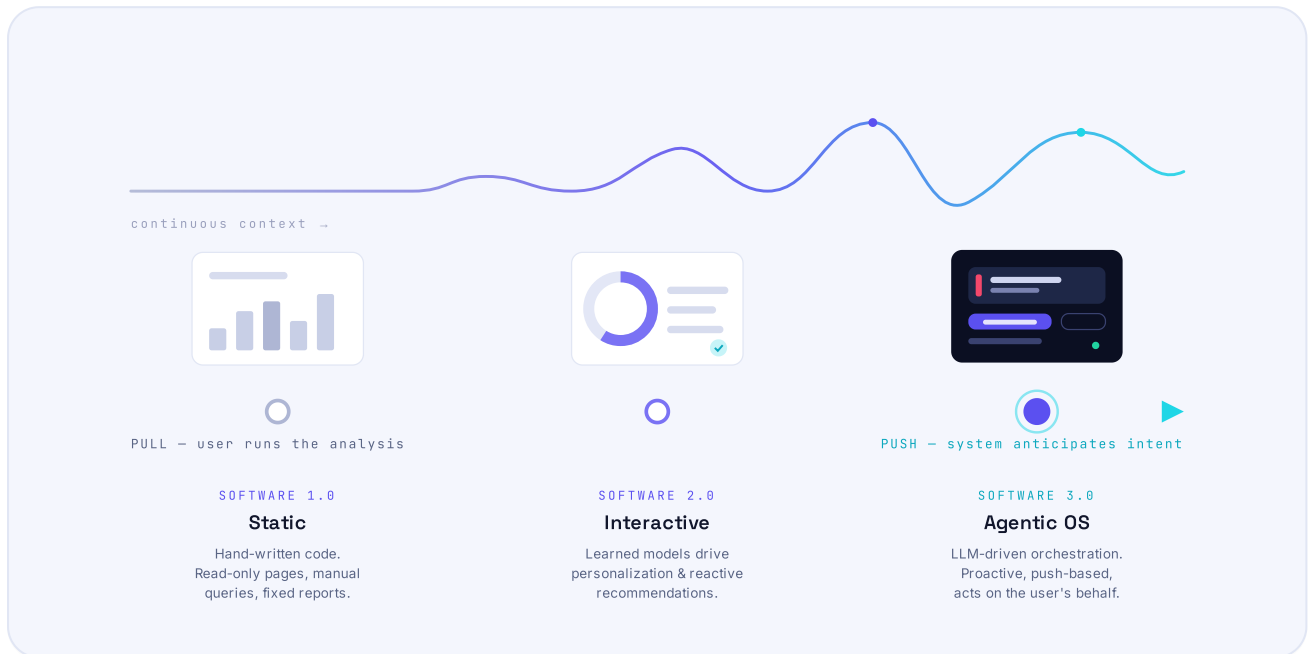


FIG. 01

The software evolution timeline. Framing follows Karpathy's Software 1.0 / 2.0 / 3.0 progression. The signal line at top depicts the growing volume of continuous context that makes each successive era's proactivity possible. ^[1]

Architecting the Contextual Data Model

Turning disparate, real-time streams into a living user profile

A system that anticipates needs cannot be fed by data sitting in isolated silos. It requires a unified substrate that continuously ingests unstructured signals — behavioral logs, third-party API data, historical actions, real-time market or sensor feeds — and fuses them into a single, living representation of the user.

a16z identifies the bottleneck precisely: most enterprise value is trapped in *messy*, multimodal data — PDFs, logs, screenshots, transcripts — and the limiting factor for AI products is now **data entropy**, the steady decay of freshness and structure in that unstructured universe. ^[2] Winning teams treat clean, vectorized, continuously-updated context as core infrastructure, not an afterthought. Two architectural moves define the model:

From personas to real-time state evaluation

Static personas ("the budget-conscious millennial") freeze a person into a stereotype. A contextual system instead evaluates the user's **real-time state** — distinguishing someone idly checking a balance from someone anxiously reacting to a market dip, and responding differently. Karpathy's caution applies directly: today's models show "jagged intelligence" and have no long-term memory of their own, so the surrounding system must supply the context, guardrails, and feedback loops that keep behavior grounded. ^[1]

From rigid schemas to a continuous context loop

Rather than forcing every signal into a brittle SQL table defined months in advance, the platform ingests data into a shared vector space and maintains a continuously-updated profile of the user's trajectory. Anthropic describes the foundational unit of these systems as the **augmented LLM** — a model enhanced with retrieval, tools, and memory, able to pull the right context and act on it. ^[4] That augmented core is what turns a static profile into a live one capable of proactive intervention.

A personal-finance app stops being a list of articles and a few charts. It becomes a system that models an individual's complete wealth trajectory — in real time.

CASE STUDY — THE FINANCIAL OPERATING SYSTEM

That reframing is the prerequisite for every proactive feature that follows. Without a living context model, "anticipation" is just a guess; with one, it becomes a defensible, improving capability.

How unstructured streams become anticipation

Raw, unstructured signals flow through an ingestion and vectorization layer into a **continuous context loop** — a living profile that is constantly re-evaluated — and out as proactive action. Crucially, the loop bypasses the rigid, predefined SQL schema that would otherwise freeze the model in place.

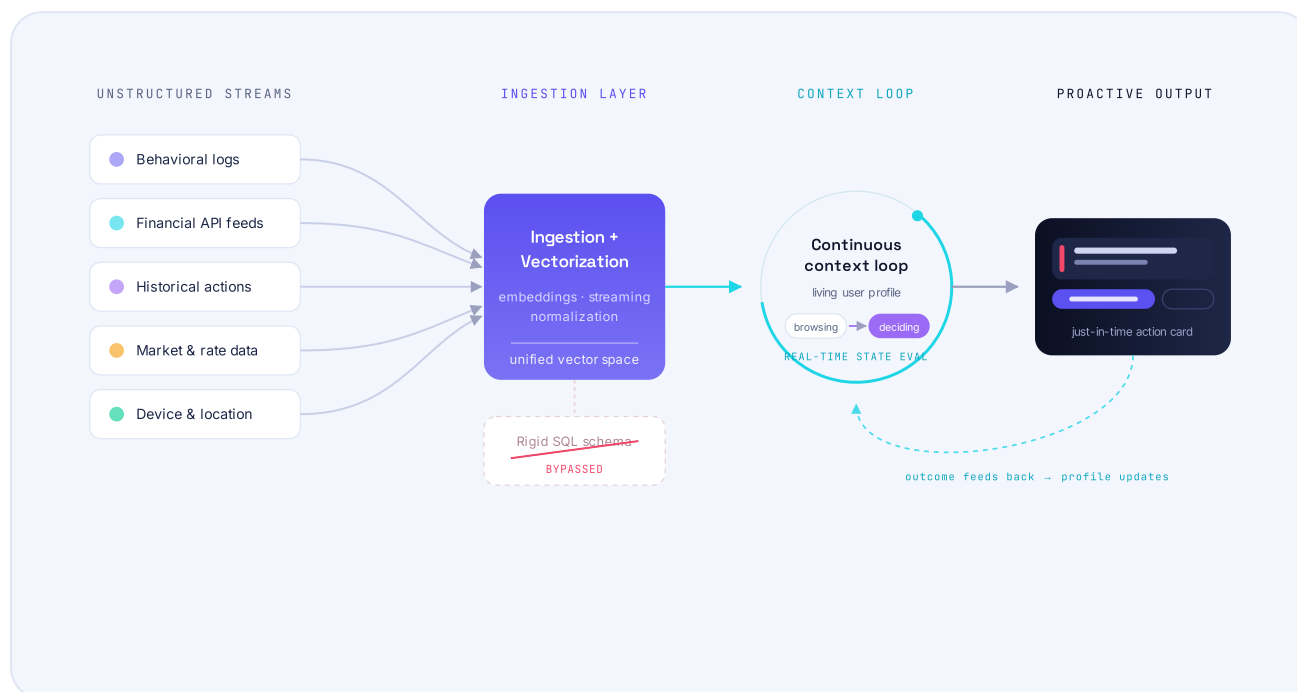


FIG. 02 The contextual ingestion engine. Unstructured streams are embedded into a shared vector space and held in a continuously re-evaluated loop, sidestepping rigid schemas. The augmented-LLM pattern sits at the core of the loop. [4]

The Proactive UI/UX Shift

From read-only to act-now

This is where predictive analytics become a product. The interface moves from menus the user must navigate to adaptive, context-aware cards that surface exactly when they are actionable. NN/g calls the discipline behind this **outcome-oriented design** — orchestrating the experience around the user's goal, with the interface generated to fit each user's exact context. ^[5]

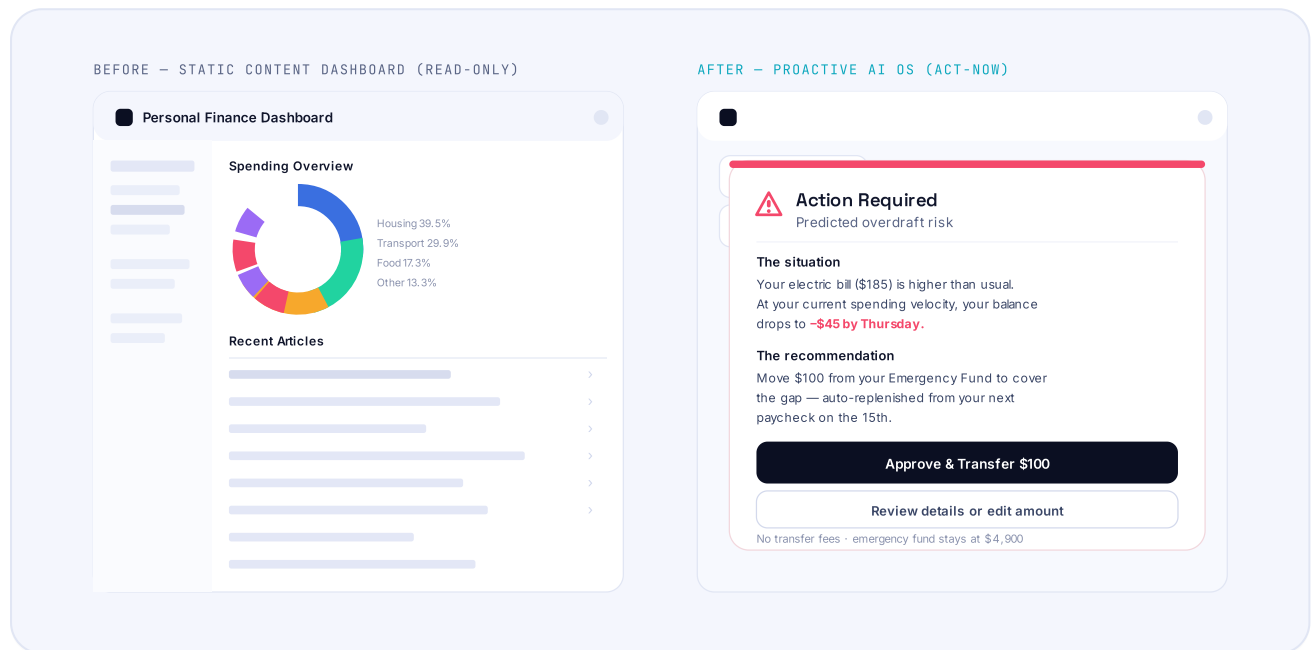


FIG. 03 Before vs. after. Left: a read-only dashboard that hands the user raw data and a list of generic articles. Right: a proactive AI OS surfacing one clear narrative insight, a predictive risk alert, and a single-click execution path.

Scripting a predictive risk-mitigation alert

A complete interaction for one high-value moment in a financial AI OS. **The scenario:** the contextual model detects that, given spending velocity and an unusually high upcoming utility bill, the user's checking account is on track to overdraw within 48 hours. Every word below is design material — active voice, the action keeping the same name from button to confirmation, failures that instruct rather than apologize. ^[5]

1

The contextual push notification

GOAL — interrupt with high signal, low anxiety. State the problem and the path to resolution in one line.

 **Cash-flow alert:** your upcoming utility bill (\$185) may overdraw your account Thursday. Tap to shift funds and avoid fees.

2

The dynamic cockpit view

GOAL — on open, an "Action Required" card sits above the home screen. The user never hunts for a budgeting tab.

Predicted overdraft risk. Your electric bill is higher than usual this month. At your current spending velocity, your balance drops to **-\$45 by Thursday**.

Recommendation: move \$100 from your Emergency Fund to cover the gap — we'll replenish it automatically from your next paycheck on the 15th.

3

The resolution panel — one-click execution

GOAL — the AI has drafted the workflow (Tier 2: Copilot); the human only approves. Insight → resolution, zero friction.

Approve & Transfer \$100

Review details or edit amount

No transfer fees. Your emergency fund stays at a healthy \$4,900.

4

The trust feedback loop

GOAL — confirm the system did exactly what it promised, and nothing more. Reinforce psychological safety.

 **Handled.** \$100 transferred — your account is safe from overdraft fees. A reminder is set to replenish on the 15th.

Engineering Trust & the Autonomy Matrix

Moving users from manual control to delegated automation

Proactive workflows only succeed if users trust the system's judgment. The central design problem of agentic software is psychological: how to move someone from manual control to delegated automation without ever making them feel they've lost the wheel. The answer, borrowed directly from Karpathy's **autonomy slider**, is to make autonomy a dial, not a switch — his analogy is the Iron Man suit: augmentation you control, handing off more as trust is earned. ^[1] Anthropic reinforces the same posture: keep humans in the loop, prefer the simplest system that works, make the agent's reasoning visible, and put hard guardrails around what it can do. ^[4] Lenny Rachitsky frames the arc in product terms — you promote an agent from *intern* to *trusted teammate* only after it proves itself on low-risk, well-scoped tasks. ^[6]

THE AUTONOMY SLIDER



Tier & rows	TIER 1 Assistant	TIER 2 Copilot	TIER 3 Agent
AI RESPONSIBILITY	Surfaces insights & data	Drafts workflows & actions	Autonomous execution
HUMAN RESPONSIBILITY	Executes all actions	Reviews & approves	Monitors via exception logs
TRUST MECHANISM	Explicit "Confirm" prompts	"Undo" windows + explanation	Transparent audit ledger

Users begin at Tier 1, confirming every action. As the system proves reliable, it graduates to Tier 2 — where the UX script above lives — and ultimately to Tier 3, where trusted micro-transactions execute autonomously, visible through transparent exception logs rather than constant prompts. The non-negotiable across all three tiers is **transparency**: a "Why did the system do this?" explanation must be built into the workflow, not bolted on. The moment reasoning becomes a black box, trust collapses — and with it, every proactive feature built on top of it.

FIG. 04 The AI autonomy matrix. A tiered model that pairs each division of labor with the specific trust mechanism it requires.

The Transformation Playbook

Metrics & strategy for an AI transformation

Transforming a platform demands new instrumentation. The old metrics actively mislead: if the AI is doing its job, the user should spend **less** time in the app, not more. "Time in app" and "daily active users" measure friction as if it were value. Three categories replace them.

01 Value, not engagement

02 Unit economics of intelligence

03 Guarding against drift

Value, not engagement. *Time to Value* (TTV) measures how fast a user resolves a high-friction problem — the five-second overdraft fix, not the thirty-minute spreadsheet session. *Automated Task Completion Rate* tracks the share of AI-suggested workflows accepted without edits — a direct read on how well the system models intent.

Unit economics of intelligence. Every proactive feature carries an inference cost, so PMs now own a real tradeoff between API latency, operational cost, and user value. a16z notes that mature teams "right-size" their models — reserving frontier models for high-stakes reasoning and routing routine work to leaner, cheaper ones. ^[2] Managing that balance is now part of the role, not an infrastructure footnote.

Guarding against drift. Real-world behavior changes, and when it does the predictive models underneath quietly degrade — distribution drift. Standing up continuous evaluation loops to catch that decay before it reaches users is a core product-safety responsibility. Anthropic's discipline of treating failures as logged, reviewable learning opportunities is the operating model here. ^[4]

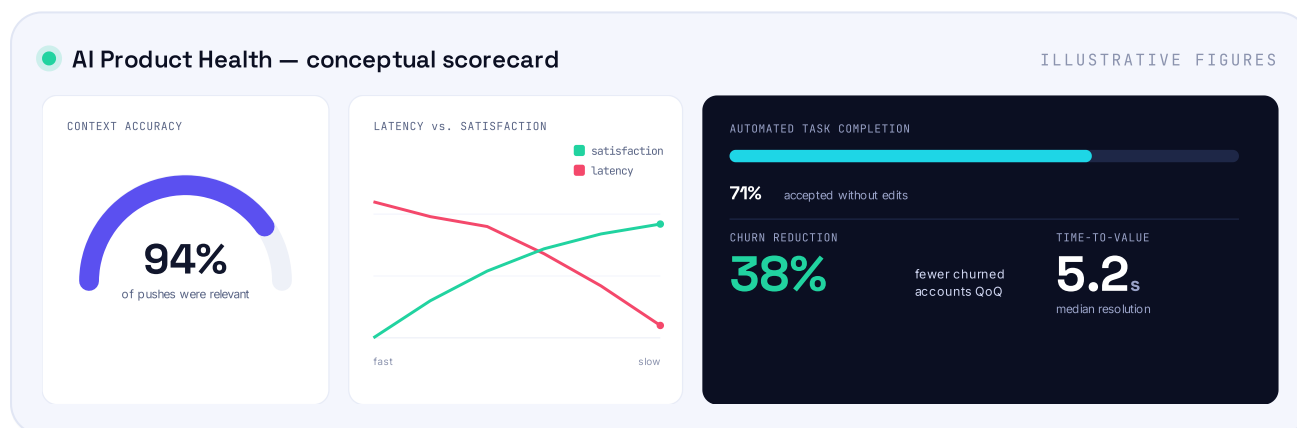


FIG. 05 The AI product health dashboard. A conceptual cockpit a PM watches instead of a vanity-metrics funnel: context accuracy, latency against satisfaction, automated completion, and churn-reduction ROI. Figures illustrative.

Becoming an Agentic Product Manager

Stop designing pages. Start designing ecosystems.

The era of designing static pages is ending. The product manager's job is no longer to lay out screens a user must operate; it is to design an **ecosystem** — a continuous loop of context, prediction, action, and trust that partners with the user toward an outcome.

a16z's prediction for the agentic era is that the passive *system of record* loses primacy to a dynamic *system of intelligence*, where the interface becomes the place work actually happens. ^[2] Building that system means combining all four disciplines in this guide.

CH. 01 — DATA

Rigorous context architecture

Ingest unstructured streams into a living, continuously-evaluated user profile.

CH. 02 — DESIGN

Proactive, humane interfaces

Replace read-only dashboards with adaptive, act-now cards and clean copy.

CH. 03 — TRUST

An earned, transparent autonomy model

Move users up the autonomy slider as reliability is proven and explained.

CH. 04 — METRICS

A new measurement discipline

Optimize time-to-value and completion, not time-in-app; guard against drift.

Stop designing pages. Start designing the operating system that renders pages unnecessary. That is the shift from content to context — and it is the defining work of the next generation of software.

THE AGENTIC MANDATE

Once you have seen what a proactive co-pilot can do — surface the risk, draft the fix, and let you approve it in a single tap — there is no going back to the dashboard that made you do the work yourself.

Recognized voices in AI product & agentic systems

The thesis of this ebook synthesizes guidance from industry experts and research. Each numbered source below maps to the in-text citations.

1

Andrej Karpathy — “Software Is Changing (Again)” / Software 3.0

YC AI Startup School, June 2025. Source of the Software 1.0/2.0/3.0 framing, the LLM-as-operating-system analogy, the “autonomy slider,” and the “jagged intelligence” & memory limitations cited throughout.

<https://www.youtube.com/watch?v=LCEmiRjPEtQ>

2

Andreessen Horowitz (a16z) — Big Ideas 2026: The Agentic Interface

a16z partners on intent-to-execution, the agent layer, “system of record → system of intelligence,” data entropy, and model right-sizing. December 2025.

<https://a16z.com/podcast/big-ideas-2026-the-agentic-interface/> · <https://a16z.com/newsletter/big-ideas-2026-part-1/>

3

Jakob Nielsen, Nielsen Norman Group — “AI: First New UI Paradigm in 60 Years”

The shift from command-based to intent-based, outcome-specification interaction. June 2023.

<https://www.nngroup.com/articles/ai-paradigm/>

4

Anthropic — “Building Effective Agents”

The workflows-vs-agents distinction, the augmented-LLM building block (retrieval, tools, memory), guardrails, simplicity, transparency, and human-in-the-loop design.

<https://www.anthropic.com/engineering/building-effective-agents>

5

Nielsen Norman Group — “Generative UI and Outcome-Oriented Design”

Designing around user goals and final outcomes; interfaces generated to fit each user's exact context. Basis for the proactive UI/UX and copywriting discipline.

<https://www.nngroup.com/articles/generative-ui/>

6

Lenny Rachitsky — AI & the future of product management

“How AI Will Impact Product Management” (PM as editor of super-intelligent suggestions) and the guide to AI agents for PMs (scoping agents, intern → trusted teammate).

<https://www.lennysnewsletter.com/p/how-ai-will-impact-product-management> · <https://www.lennysnewsletter.com/p/make-product-management-fun-again-9f6>

ALSO WORTH READING

OpenAI — **A Practical Guide to Building Agents** (agent design & guardrails). · Anthropic — **Constitutional AI** (the basis for “constitutional guardrails” in UX). · NN/g — **A Research Agenda for Generative AI in UX** (designing for both humans and agents). These extend the patterns referenced above.

